HAI802I Algorithmique Avancée — Examens

Ivan Lejeune

11 avril 2026

Table des matières

1 Complexité : NP-complétude et polynomialité 2
2 Approximation 3

Liste des Algorithmes

1 Algorithme pour le Max 3D Matching 3
2 Algorithme d'approximation pour le problème de Weighted Vertex Cover . . 4

Liste des Problèmes

1 Vertex Cover (VC) 2
2 Connected Vertex Cover (CVC) 2
3 Dominating Set (DS) 2
4 3-Sat 2
5 4-Sat 2
6 Max 3-Dimensional Matching (MAXDM) 3
7 Weighted Vertex Cover (WVC) 3
8 Le problème d'ordonnancement $P \mid \text{prec} \mid C_{\max}$ 5

1 Complexité : NP-complétude et polynomialité

Exercice 1 Complexité autour du Vertex Cover. On considère une variante du problème classique du Vertex Cover. Dans cette variante, intitulée Connected Vertex Cover, l'ensemble des sommets S choisis doit être connexe.
Pour rappel, voici la définition des deux problèmes :

Problème 1 – Vertex Cover (VC)
Input: Un graphe $G = (V, E)$ et un entier k
Question: Existe-t-il un ensemble de sommets $S \subseteq V$ de taille au plus k tel que chaque arête de E est adjacente à au moins un sommet de S ?

Problème 2 – Connected Vertex Cover (CVC)
Input: Un graphe $G = (V, E)$ et un entier k
Question: Existe-t-il un ensemble de sommets $S \subseteq V$ de taille au plus k tel que chaque arête de E est adjacente à au moins un sommet de S et que S est connexe ?

Problème 3 – Dominating Set (DS)
Input: Un graphe $G = (V, E)$ et un entier k
Question: Existe-t-il un ensemble de sommets $S \subseteq V$ de taille au plus k tel que chaque sommet de V est soit dans S soit adjacent à un sommet de S ?

1. Montrer que Connected Vertex Cover est NP-complet.
2. Montrer que Vertex Cover reste NP-complet pour un graphe de degré au plus 3.
   Aide : Transformer les sommets de degré 4.
3. Montrer que Dominating Set est NP-complet. La preuve se fera à partir de Vertex Cover
4. Quelle est la complexité du problème Vertex Cover dans les graphes bipartis ?
5. Quelle est la complexité du problème Vertex Cover dans les graphes avec un degré maximum de 2 ?

Solution.

1. Il suffit de rajouter un sommet α qui domine le graphe. Alors, tout vertex cover doit forcément contenir α et sera donc connexe
2. A remplir
3. Il faut transformer chaque arête en triangle. Alors on obtient bien une réduction de Vertex Cover à Dominating Set.
4. Le problème est polynomial dans les graphes bipartis, il suffit de faire un couplage maximum.
5. Le problème est polynomial dans les graphes de degré maximum 2 car ils correspondent à des chemins et des cycles.

Exercice 2 Complexité autour du Sat. Prouver que le problème 4-Sat est NP-complet. Vous pouvez utiliser le fait que 3-Sat est NP-complet pour obtenir ce résultat.
On rappelle les formulations de ces deux problèmes :

Problème 4 – 3-Sat
Input: Une formule booléenne φ en forme normale conjonctive (CNF) avec des clauses de taille 3
Question: Existe-t-il une affectation des variables qui satisfait φ ?

Problème 5 – 4-Sat

Input: Une formule booléenne φ en forme normale conjonctive (CNF) avec des clauses de taille 4

Question: Existe-t-il une affectation des variables qui satisfait φ ?

Solution. On peut reprendre la transformation énoncée dans les tds, on convertit les sommets de degré 4 en sommets de degré 3 en introduisant un nouveau sommet v au centre et x, y aux bords.
On connecte v à x et y, puis on connecte x et y à deux des quatre voisins du sommet de degré 4.

2 Approximation

Exercice 3 Autour du Max 3-Dimensional Matching.
On rappelle la formulation du problème comme suit :

Problème 6 – Max 3-Dimensional Matching (MAXDM)

Input: Un ensemble $S \subseteq X \times Y \times Z$ de triplets.

Question: Trouver un couplage de cardinalité maximum, i.e. un ensemble $M \subseteq S$ où toute paire de triplets de M n'ont aucun élément en commun.

et on propose l'algorithme d'approximation suivant :

Algorithme 1 – Algorithme pour le Max 3D Matching

Input: Un ensemble $S \subseteq X \times Y \times Z$ de triplets.
Output: Un couplage $M \subseteq S$
$M \leftarrow \varnothing$
for chaque triplet $(x, y, z) \in S$ **do**
  if (x, y, z) ne contredit pas M **then**
    $M \leftarrow M \cup \{(x, y, z)\}$
return M

1. Donner la complexité de l'algorithme 1
2. Montrer que l'algorithme 1 est 3-approché.
3. Trouver une instance pour laquelle la borne de trois est atteinte.
   Aide : Considérer un ensemble de $4h$ triplets avec $h \in \mathbb{N}$. L'algorithme prendra que h triplets tandis que la solution optimale en prendra $3h$.

Solution.

1. L'algorithme a une complexité de $O(n^2)$.
2. L'algorithme ne peut pas etre mieux que 3 approché, car on a un exemple de cas 3-approché. De plus, a chaque étape on dégage au pire 3 triplets de la solution optimale, alors on a forcément une approximation de facteur 3.
3. Il suffit de considérer 4 triangles dont 3 qui connectent à un sommet du triangle restant.

Exercice 4 Autour de Weighted Vertex Cover.
On considère dans la suite le problème du Weighted Vertex Cover formulé de la manière suivante :

Problème 7 – Weighted Vertex Cover (WVC)

Input: Un graphe $G = (V, E, \omega)$ où $\omega : V \to \mathbb{R}^+$ est une fonction de poids sur les sommets, et un entier k.
Question: Existe-t-il un ensemble de sommets $V' \subseteq V$ qui couvre toutes les arêtes de G et dont le poids total est au plus k ?

et on propose l'algorithme d'approximation suivant :

Algorithme 2 – Algorithme d'approximation pour le problème de WEIGHTED VERTEX COVER

On commence par résoudre la relaxation du problème de WEIGHTED VERTEX COVER.
Soit $V_0 = \{v \in V \mid x_v^* = 0\}, V_{1/2} = \left\{v \in V \mid x_v^* = \frac{1}{2}\right\}, V_1 = \{v \in V \mid x_v^* = 1\}$.
L'ensemble $V_{1/2}$ utilise k couleurs pour être coloré par une coloration propre. Soit C la classe de couleur c ayant le maximum de sommets.
Retourner $V' = V_1 \cup (V_{1/2} \setminus C)$.

---

1. Rappeler le programme linéaire en nombres entiers pour le problème du WEIGHTED VERTEX COVER.
2. Montrer que $V_1 \cup \left(V_{1/2} \setminus C\right)$ forme un vertex cover.
3. Montrer que le coût de la solution retournée par l'algorithme 2 est de

$$(2 - 2/k) \cdot \sum_{v \in V} x_v^* w_v$$

Solution.

1. A remplir
2. Tous les sommets de couleur c forment un stable, donc ils sont reliés à $V_{1/2} \setminus C$ par des arêtes. Alors, V_1 couvre les arêtes qui ne sont pas couvertes par $V_{1/2} \setminus C$ et $V_{1/2} \setminus C$ couvre les arêtes qui ne sont pas couvertes par V_1.
3. On a le cout de la solution qui est :

$$\sum_{v \in V_1} x_v^* w_v + \sum_{v \in V_{1/2} \setminus C} x_v^* w_v + \sum_{v \in V_0} x_v^* w_v$$

Par le choix de C on a aussi :

$$\sum_{v \in C} 2x_v^* w_v \geq \frac{1}{k} \sum_{v \in V_{1/2}} 2x_v^* w_v$$
$$\iff -\sum_{v \in C} 2x_v^* w_v \leq -\frac{1}{k} \sum_{v \in V_{1/2}} 2x_v^* w_v$$

Alors

$$
\begin{aligned}
&\sum_{v\in V_1} x_v^* w_v + \sum_{v\in V_{1/2}\setminus C} x_v^* w_v + \sum_{v\in V_0} x_v^* w_v \\
\leq &\sum_{v\in V_1} x_v^* w_v + \sum_{v\in V_{1/2}\setminus C} 2x_v^* w_v + \sum_{v\in V_0} 2x_v^* w_v \\
\leq &\sum_{v\in V_0\cup V_1} x_v^* w_v + \sum_{v\in V_{1/2}} 2x_v^* w_v - \frac{1}{k}\sum_{v\in V_{1/2}} 2x_v^* w_v \\
\leq &\ 2\cdot\sum_{v\in V} x_v^* w_v - \frac{2}{k}\sum_{v\in V_{1/2}} x_v^* w_v \\
\leq &\ 2\cdot\sum_{v\in V} x_v^* w_v - \frac{2}{k}\sum_{v\in V} x_v^* w_v \\
\leq &\ (2-2/k)\cdot\sum_{v\in V} x_v^* w_v \\
\leq &\ (2-2/k)\cdot OPT_{ILP}
\end{aligned}
$$

Exercice 5 Autour du problème d'ordonnancement avec contraintes de précédence.
On rappelle que le problème $P \mid \text{prec} \mid C_{\max}$ est défini de la manière suivante :

Problème 8 – Le problème d'ordonnancement $P \mid \text{prec} \mid C_{\max}$
Input: n tâches de durées respectives $p_1, p_2, \ldots, p_n$ et $\Longleftarrow (I, U)$ une relation résumant les contraintes de précédence entre les tâches.
Question: Ordonnancer ces tâches sur m machines identiques en une durée totale minimale notée $C_{\max}$ tout en respectant les contraintes de précédence.

1. On veut montrer que pour n'importe quel ordonnancement de liste LS on a

$$\frac{C_{\max}^{LS}}{C_{\max}^{*}} \leq 2 - \frac{1}{m}$$

   (a) Pour cela, considérons la tache T_{i_1} dont la fin d'exécution coincide avec $C_{\max}^{LS}$. Soit $T_{i_2} = T_j \setminus T_j \in \Gamma^-(T_{i_1}) \wedge C_j = \max_{T_j\in\Gamma^-(T_{i_1})} C_i$ où $C_i = t_i + p_i$ désigne la fin d'exécution de la tache i et $\Gamma^-(T_{i_1})$ désigne l'ensemble des prédécesseurs de i_j.
   En procédant ainsi de suite, on a $T_{i_k} = T_j \setminus T_j \in \Gamma^-(T_{i_{k-1}}) \wedge C_j = \max_{T_j\in\Gamma^-(T_{i_{k-1}})} CT_i$.
   On appellera P le chemin constitué de ces taches $T_{i_1}, T_{i_2}, \ldots, T_{i_k}$.
   Est-ce qu'il existe des processeurs (ou machines) inactifs entre la fin d'exécution d'un prédécesseur de T_{i_1} et le début d'exécution de T_{i_1} ?
   Est-ce qu'il existe des processeurs (ou machines) inactifs entre la fin d'exécution d'un prédécesseur de T_{i_k} et le début d'exécution de $T_{i_k}, \forall k$?
   Que pouvons-nous conclure sur les dates de début d'exécution des taches appartant au chemin P ?

   (b) A partir de l'équation suivante, conclure sur le ratio :

$$C_{\max}^{LS} = \frac{T_{\text{seq}} + T_{\text{idle}}}{m}$$

   (c) Pouvons-nous appliquer le même algorithme au problème $P \mid \text{prec}, r_j \mid C_{\max}$, c'est-à-dire le même problème décrit ci-dessus mais avec le début des taches T_j qui ne peut se faire qu'après la date de disponibilité r_j ?

2. Montrer que la borne supérieure est atteinte. Pour cela, considérer l'instance suivante :
   - m taches indépendantes,

- la tache m précède les taches $m+1, m+2, \ldots, 2m$,
- la tache $2m+1$ suit les taches $m+1, m+2, \ldots, 2m$.

Les durées des taches sont les suivantes :

- $p_1 = p_2 = \cdots = p_{m-1} = m-1$,
- $p_m = \varepsilon$,
- $p_{m+1} = p_{m+2} = \cdots = p_{2m} = 1$,
- $p_{2m+1} = m-1$.

Solution.

1. Sur le ratio d'ordonnancement de liste :
   (a) On obtient un chemin incompressible, les dates d'exécution ne peuvent être avancées.
   (b) On a bien
   $$m \cdot C_{\max} = T_{\text{seq}} + T_{\text{idle}}$$
   d'où
   $$\begin{aligned} C_{\max} &= \frac{T_{\text{seq}} + T_{\text{idle}}}{m} \\ &\leq \frac{T_{\text{seq}} + (m-1)\sum_{i=1}^{n} P_i p}{m} \\ &= OPT + \left(1 - \frac{1}{m}\right) OPT \\ &= \left(2 - \frac{1}{m}\right) OPT \end{aligned}$$